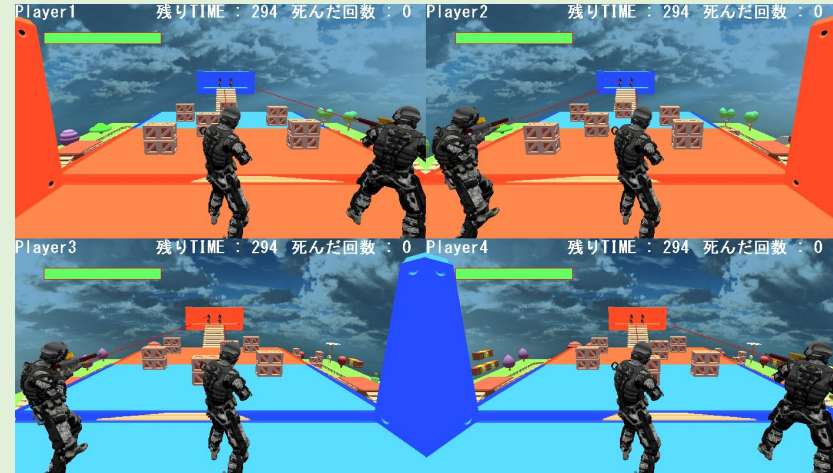
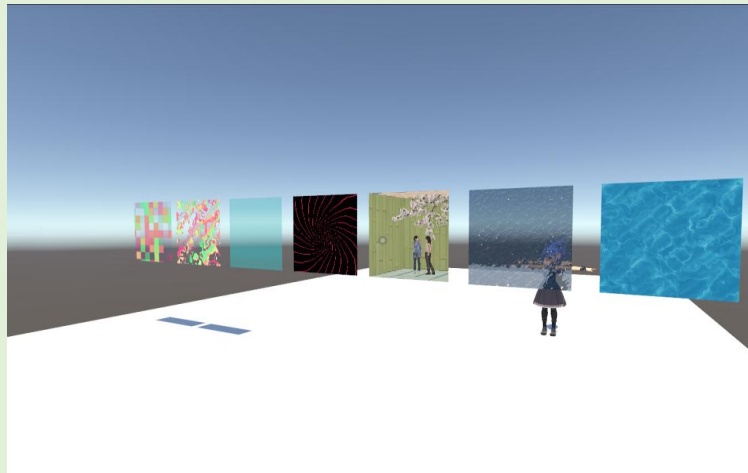


PORTFOLIO

時枝 龍太

TOKIEDA RYUTA



Profile

時枝 龍太

IT／エンターテインメント企業 エンジニア

自己紹介

学生時代はC++を中心に、ゲーム制作を軸として開発してきたエンジニアです。
DXライブラリや Unity を用いて、2D／3D ゲームを多数制作し、クォータニオン・ベクトル・行列演算を用いたキャラクター制御やカメラ制御、衝突判定を実装してきました。
弾幕制御、線分ベースの当たり判定、レイトレーシングなど、基礎技術を理解した上での実装を大切にしています。

社会人になってからは、Unreal Engine を用いた画像認識シミュレーション開発や、
Web(LP作成・保守・運用)、AIエージェント作成などにも携わり、エンタメを支える技術を幅広く経験してきました。
また、業務の傍ら帰宅後には HLSL を用いたグラフィック表現の学習や C# の勉強に継続的に取り組み、技術の幅を広げています。

プレイヤー体験をシステムで支えるエンジニアとして、エンターテインメント分野での開発に携わりたいと考えています。

Demo

HD2D表現



開発環境:Unity

勉強期間:2か月

使用言語:C# / HLSL

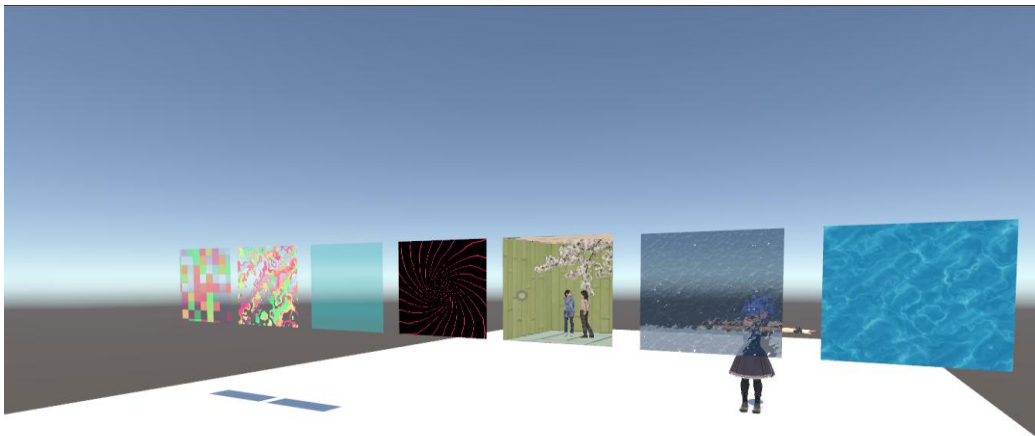
ジャンル:HD2Dデモ



HD-2D 表現に魅力を感じ、表現方法の実装に取り組みました。

表現手法がわからない部分については、YouTube や技術記事を参考にしながら検証を重ね、影Shader、衝突判定、ポストプロセスを実装しています。また、カメラとプレイヤーの間にオブジェクトが存在する場合に視認性が損なわれる課題に対し、カメラ方向からプレイヤーに重なっているオブジェクトを判定し、半透明化する処理を実装しました。これにより、HD-2D 特有の奥行き感を保ちつつ、プレイヤーの視認性を確保しています。

グラフィック表現



VRChat内で、綺麗さや驚きのある表現を実現したいと考え、Shader 制作の学習を進めています。

まずは Unlit Shader、UV スクロール、ディゾルブ Shader などの基礎的な表現から取り組み、現在は Interior Mapping や Toon Shader の自作にも挑戦しています。

ホログラム Shader では、透明表現によって衣服の内部などが透けて見えてしまう問題に対し、Z バッファのみを書き込むパスと通常描画パスを分けた 2Pass 構成を採用し、透明感を保ちつつ、オブジェクトの前後関係が破綻しない、見た目と実用性を両立した Shaderを実現しています。

不明点が多い分野ではありますが、AI との対話や技術資料を活用しながら、「どうすれば実現できるか」「どこが問題になり得るか」を一つずつ整理し、試行錯誤を重ねながら継続的に学習を進めています。

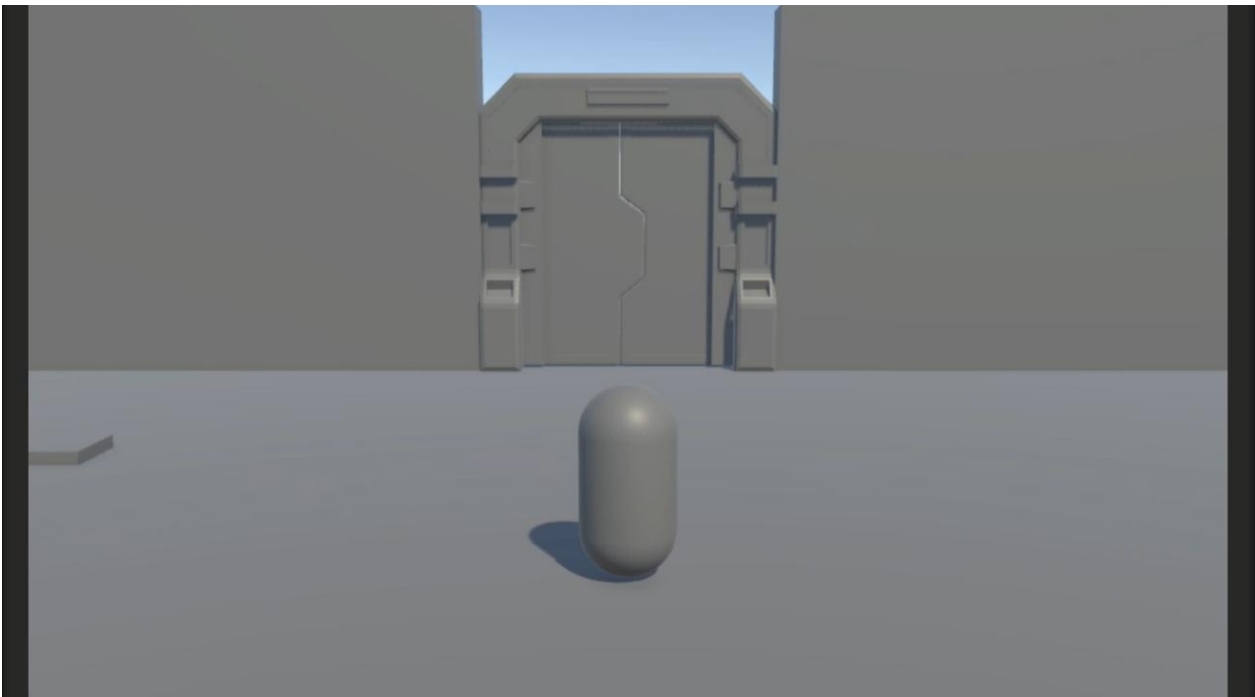
開発環境:Unity

使用言語:HL SL

勉強期間:5か月

ジャンル :Shader グラフィック表現

時間巻き戻しアクション



動画はサイトから下記サイトで閲覧可能です。

<https://tokimarusuisann.github.io/portfolio/>

Unityで制作中の時間巻き戻しと残像(Ghost)を用いた3Dアクションです。

プレイヤーは最大5秒間、過去の行動を巻き戻すことができ、巻き戻した行動は実体を持つ残像として再生され、床スイッチや扉などのギミック解決に利用します。

三人称視点での移動・ジャンプ・巻き戻し操作を実装し、プレイヤーの行動を FixedUpdate ベースで記録し、リングバッファを用いて一定時間分を保持しています。

巻き戻し時は入力を無効化した上で記録データを逆順に適用し、安定した時間操作を実現しています。また、残像は同じ記録データを再利用して再生しており、物理的な当たり判定を持つ実体としてプレイヤーやギミックと干渉します。

制作期間は1週間のため見た目は最小限ですが、3D空間における状態管理・再生処理・責務分離を重視し制作しています。

開発環境:Unity

使用言語:C#

勉強期間:一週間

ジャンル :時間巻き戻しアクション

学生時代作品

Exciteレース



学校内でXboxのレース用コントローラーを見つけたので好奇心で制作した作品です。

1～2人プレイです。

1人：タイムアタック 2人：VSレース

ブレーキ、シフトチェンジをうまく使い競い合うゲームです
ゲームセンターにある頭文字Dアーケードや湾岸ミッドナイト
アーケードを参考にゲームを制作いたしました。



制作時期：三年前期

開発環境：DXライブラリ

使用言語：C++

制作期間：3か月

ジャンル：レースゲーム

カメラ制御

通常の追従カメラでは疾走感が欠ける問題があったため、クォータニオンの Slerp（球面線形補間）を用いたカメラ回転制御を実装しました。

現在のカメラ回転とターゲットの回転を補間し、その結果を基準に相対位置を算出することで、プレイヤーの向きに自然に追従しつつ、滑らかでスピード感のあるカメラ挙動を実現しています。

```
199 void Camera::SetBeforeDrawFollow(void)
200 {
201     //指定したオブジェクトのポジション
202     VECTOR targetPos = target->pos_;
203     //指定したオブジェクトの回転
204     Quaternion targetRot = target->quaRot_;
205     //今のカメラの状態
206     Quaternion nowCamera = Quaternion::Euler(0.0f, angleY_, 0.0f);
207     //今の状態のカメラからプレイヤーの回転まで球面補間
208     slerpRot_ = Quaternion::Slerp(nowCamera, targetRot, SLERP_TIME);
209     //カメラ位置と注視点を移動
210     VECTOR relativeCameraPos = slerpRot_.PosAxis(SLERP_RELATIVE_CAMERA_POS);
211     pos_ = VAdd(targetPos, relativeCameraPos);
212     VECTOR relativeTargetPos = slerpRot_.PosAxis(SLERP_RELATIVE_TARGET_POS);
213     targetPos_ = VAdd(pos_, relativeTargetPos);
214 }
215
216
217
218
219
220
```

※「Slerpを用いた回転補間による追従カメラ制御

通常追従カメラ

<https://youtu.be/wVaLajl5mxU>



球面線形補間を使った追従カメラ

<https://youtu.be/cEsm7d3B2jA>



衝突判定

高速移動時に通常のカプセル判定や円形判定では衝突のすり抜けが発生していたため、車両の移動前座標と移動後座標を用いた線分衝突判定を実装しました。移動ベクトル上でコライダーとの交差判定を行い、ヒット時には法線方向への押し戻しと減速処理を行うことで、安定した挙動と自然な衝突レスポンスを実現しています。

```
525 void Car::Collision(void)
526 {
527     //前方向
528     VECTOR forward = transformCar_.GetForward();
529     //線形補間で押し戻す
530     collisionPow_ = AsoUtility::Lerp(collisionPow_, AsoUtility::VECTOR_ZERO, COLLISION_POWER);
531     //移動先の座標
532     movedPos_ = VAdd(transformCar_.pos_, VScale(forward, speed_));
533     //押し戻した座標を足す
534     movedPos_ = VAdd(movedPos_, collisionPow_);
535     //移動前座標と移動先座標の線分で衝突判定を行う
536     for(auto collider : colliders_)
537     {
538         auto hit = MV1CollCheck_Line(collider, -1, transformCar_.pos_, movedPos_);
539         //衝突していたら
540         if(hit.HitFlag > 0)
541         {
542             auto& normal = hit.Normal;
543             //高さは無視する
544             normal = VNorm(normal);
545             normal.y = 0.0f;
546             //押し戻し処理
547             movedPos_ = VAdd(hit.HitPosition, VScale(normal, MOVED_POS_SIZE));
548             //減速処理
549             Deceleration(SPEED_DOWN_BREAK_POWER);
550         }
551     }
552     //更新
553     transformCar_.pos_ = movedPos_;
554 }
555
```

※高速移動時のトンネリング対策として線分衝突判定を実装

モデルと図形での衝突判定

<https://youtu.be/trjMWBY30is>

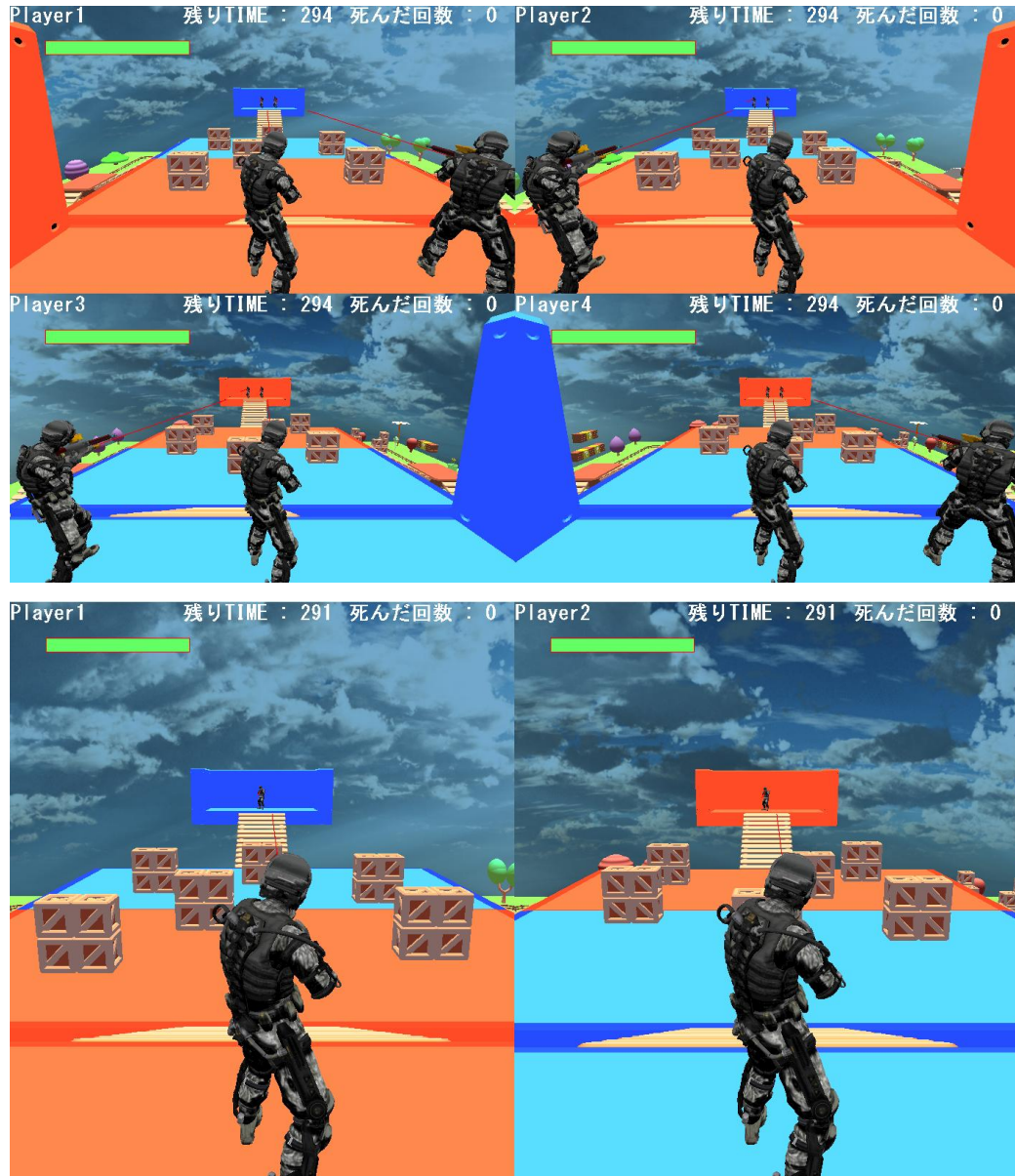


線分での衝突判定

<https://youtu.be/B6m0b01z7z4>



TPSゲーム



みんなで楽しく遊べるゲームを作りたいという思いで制作いたしました。

2～4人プレイです。
2人プレイ、3人プレイは個人戦です。
4人プレイはチーム戦です。

画面分割をし1画面で多人数遊べるようにしました。
友人に遊んでもらい意見をもらいながら制作いたしました。ボーン制御にこだわった作品です

制作時期：二年後期

開発環境：DXライブラリ

使用言語：C++

制作期間：3か月

ジャンル：TPS

このゲームを作る際、ステージに高低差があるため、プレイヤーが上下に球を打ち分けることができるようにしました、その際アニメーションを使わず実際にボーン(フレーム)を動かしました。



クォータニオンによるボーン制御

ボーンを指定して回転させる処理

```
269 MATRIX Player::MoveFrameRotateMatrix(int mHandle, int moveFrameIndex, float movePow, VECTOR rotDir)
270 {
271     //行列を初期化 (前回の行列が残ったままになりおかしくなる)
272     MV1ResetFrameUserLocalMatrix(mHandle, moveFrameIndex);
273
274     //ワールド回転をローカル回転に変換するために作った逆回転関数を呼び出し
275     Quaternion LocalReverseRot = GetRotReverseBornWorldToLocal(mHandle, moveFrameIndex);
276
277     //現在のローカル座標からワールド座標に変換する行列を得る
278     auto worldMatrix = MV1GetFrameLocalWorldMatrix(mHandle, moveFrameIndex);
279
280     //上で取ってきた情報で(大きさ、位置、回転が入っている)一応回転成分のみ抽出
281     auto worldRotMatrix = MGetRotElem(worldMatrix);
282
283     //背骨のワールドクォータニオンをゲット
284     auto worldRotQuaternion = Quaternion::GetRotation(worldRotMatrix);
285
286     //回転させたい回転量
287     auto Rot = Quaternion::AngleAxis(movePow, rotDir);
288
289     //回転させたい回転量に背骨のワールドクォータニオンを合成
290     auto MixSpineWorldQua = Rot.Mult(worldRotQuaternion);
291
292     //逆回転を合成
293     auto mixSpineLocalQua = LocalReverseRot.Mult(MixSpineWorldQua);
294
295     //初期状態の変換行列
296     auto InitMat = MV1GetFrameBaseLocalMatrix(mHandle, moveFrameIndex);
297
298     //初期状態の拡大率の取得
299     auto VecScal = MGetSize(InitMat);
300
301     //初期上の移動行列取得
302     auto VecPos = MGetTranslateElem(InitMat);
303
304     //初期状態の拡大行列の取得
305     auto MatScal = MGetScale(VecScal);
306
307     //平行移動行列の取得
308     auto MatPos = MGetTranslate(VecPos);
309
310     //Quaternionから行列へ変換
311     auto MultMat = mixSpineLocalQua.ToMatrix();
312
313     //単位行列に取得してきた行列を合成
314     MATRIX mat = MGetIdent();
315     mat = MMult(mat, MatScal);
316     mat = MMult(mat, MultMat);
317     mat = MMult(mat, MatPos);
318
319     return mat;
320 }
```

ねじれをクォータニオンで表すために、親ボーンの座標軸の方向を子ボーンの座標軸に合わせる計算(ワールド空間での回転クォータニオンをローカル空間に変換)

```
Quaternion Player::GetRotReverseBornWorldToLocal(int mHandle, int moveFrameIndex)
{
    //素のQuaternion(最終return値)
    Quaternion result = Quaternion::Identity();

    //親フレーム番号
    auto FrameParentNum = MV1GetFrameParent(mHandle, moveFrameIndex);

    //親フレームが存在する間(parentFrameが-2ではない)
    while (FrameParentNum >= 0)
    {
        //ここでワールド回転をローカル回転に変換 親の行列 (大きさ、回転、位置)
        auto LocalParentMat = MV1GetFrameLocalMatrix(mHandle, FrameParentNum);

        //親の行列をクォータニオンに変換 (回転)
        auto LocalQuaRotParent = Quaternion::GetRotation(LocalParentMat);

        //親の逆回転情報 (Inverse()) を合成してあげる、
        result = result.Mult(LocalQuaRotParent.Inverse());

        //ここで計算したフレームに親フレームがあれば回る
        FrameParentNum = MV1GetFrameParent(mHandle, FrameParentNum);
    }
}
```

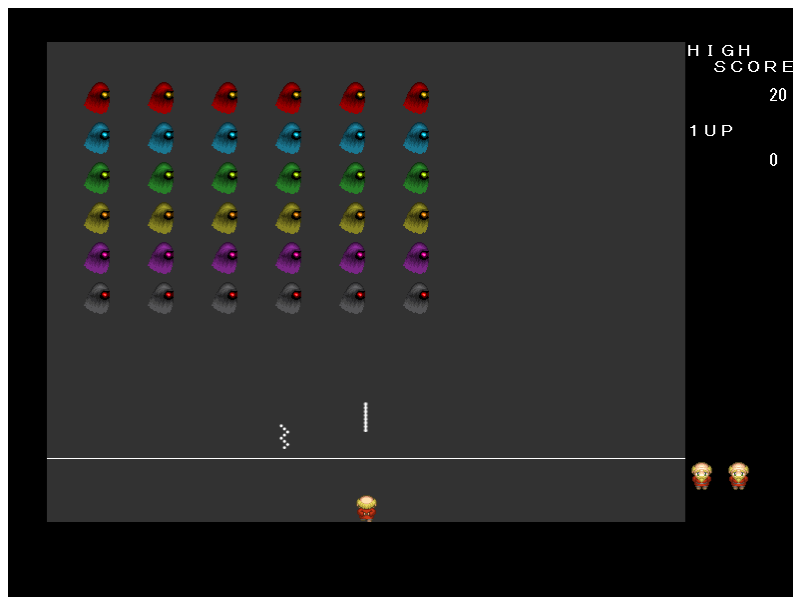
一年次



シューティング

制作時期 : 一年前期
使用言語 : C言語
使用ライブラリ : DXライブラリ

初めてのゲームプログラミングでシューティングゲームを制作いたしました。C言語の基礎や、画像の描画、衝突判定、キー入力、ゲームループなどの基本的なことを学びました。背景のスクロールは画像を2枚並べて交互に来るようにして実装しています。

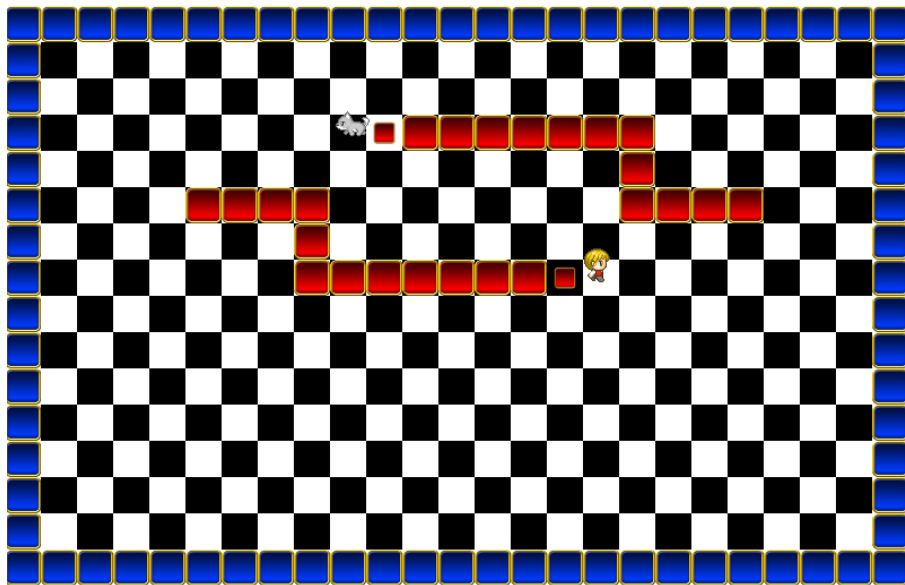


インベーダー

制作時期 : 一年前期
使用言語 : C言語
使用ライブラリ : DXライブラリ

インベーダーゲームを制作しました。文字の描画や、スコア、残機、キャラクターのアニメーション、複数の敵を配列で管理する方法などを学びました。ゲームに面白みを持たせるにサウンド、ステージ選択、ボス戦などを実装しました。

一年次



スネークゲーム

制作時期 : 一年後期
使用言語 : C++
使用ライブラリ : DXライブラリ

初めてC++言語で制作しました。カプセル化やポリモーフィズム、関数オブジェクト、シングルトン、標準テンプレートライブラリなどを学びました。当たり判定をする為にbool型の二次元配列を用意し、座標を今いるマス目に計算して勝敗判定を実装しました。

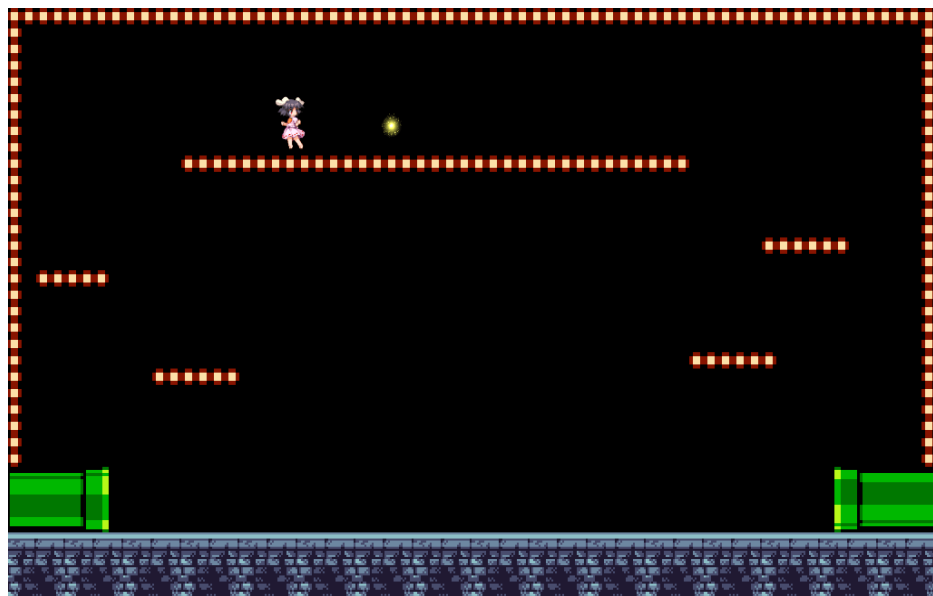
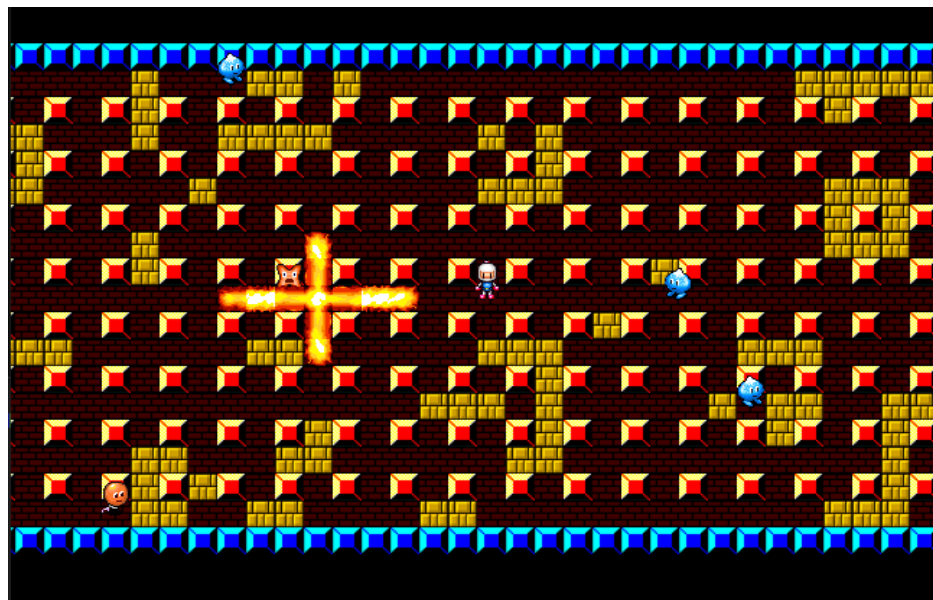


2Dアクション

制作時期 : 一年後期
使用言語 : C#
使用エンジン : Unity

初めてのUnity制作で2Dアクションを作りました。カメラの追従、敵や星のジェムをプレハブ化しコピー生成、アニメーターでモーション切り替え、敵を踏んだ時の当たり判定など実装しました。

二年次



ボンバーマン

制作時期	: 二年前期
使用言語	: C++言語
使用ライブラリ	: DXライブラリ

ステージをTiledMapEditorで制作しXML形式で読み込み実装しました。十字に伸びる爆風や誘爆の実装方法、ラムダ式や継承、STL、関数ポインタなどの使い方を学びました。

コマンドアクション

制作時期	: 二年前期
使用言語	: C++言語
使用ライブラリ	: DXライブラリ

TiledMapEditorの機能を用いて制作した衝突判定用のオブジェクトレイヤーを読み込み、そのデータを用いてプレイヤーとステージの衝突処理を実装しました。リングバッファを用いることでコマンド入力を実装し、波動拳や昇竜拳を撃つことができます。

二年次



ESCAPEゲーム

制作時期 : 二年前期
使用言語 : C#
使用エンジン : Unity

Unityで3Dアクションを作りました。
3Dゲームとしては初めての制作でしたが、C#の関数を用いてプレイヤー追跡、弾発射、アニメーション切り替え等をプログラムで実装しました。



3Dシューティングゲーム

制作時期 : 二年前期
使用言語 : C++
使用ライブラリ : DXライブラリ

C++での初めての3Dゲーム制作でした。
クォータニオンを使用した回転、バネの力を応用した時間差でカメラの追従を連動させる処理、ボス戦の前のイベントシーン等を実装しました。

二年次



3Dタワーディフェンス

制作時期 : 二年前期
使用言語 : C++
使用ライブラリ : DXライブラリ

3Dゲームの復習として、タワーディフェンスゲームを制作しました。砲台と砲身の親子関係を設定したり、砲身の角度に合わせて弾を発射する処理、楽しめるようにステージや敵を複数実装しました。



3Dアクションゲーム

制作時期 : 二年後期
使用言語 : C++
使用ライブラリ : DXライブラリ

ステージによって重力方向が変化したり傾斜の角度によって坂道を滑り落ちる処理、また、ステージ間の移動には線形補間を使いマリオギャラクシーのようなステージ移動を実装いたしました。

数学作品



数学 シューティング

制作時期	: 二年前期
使用言語	: C++
使用ライブラリ	: DXライブラリ

ベクトルから弾幕の発射角度を求め、様々な種類の弾幕を発射できるようにしました。自機狙い弾、全方位弾、ばら撒き弾等を実装しました。

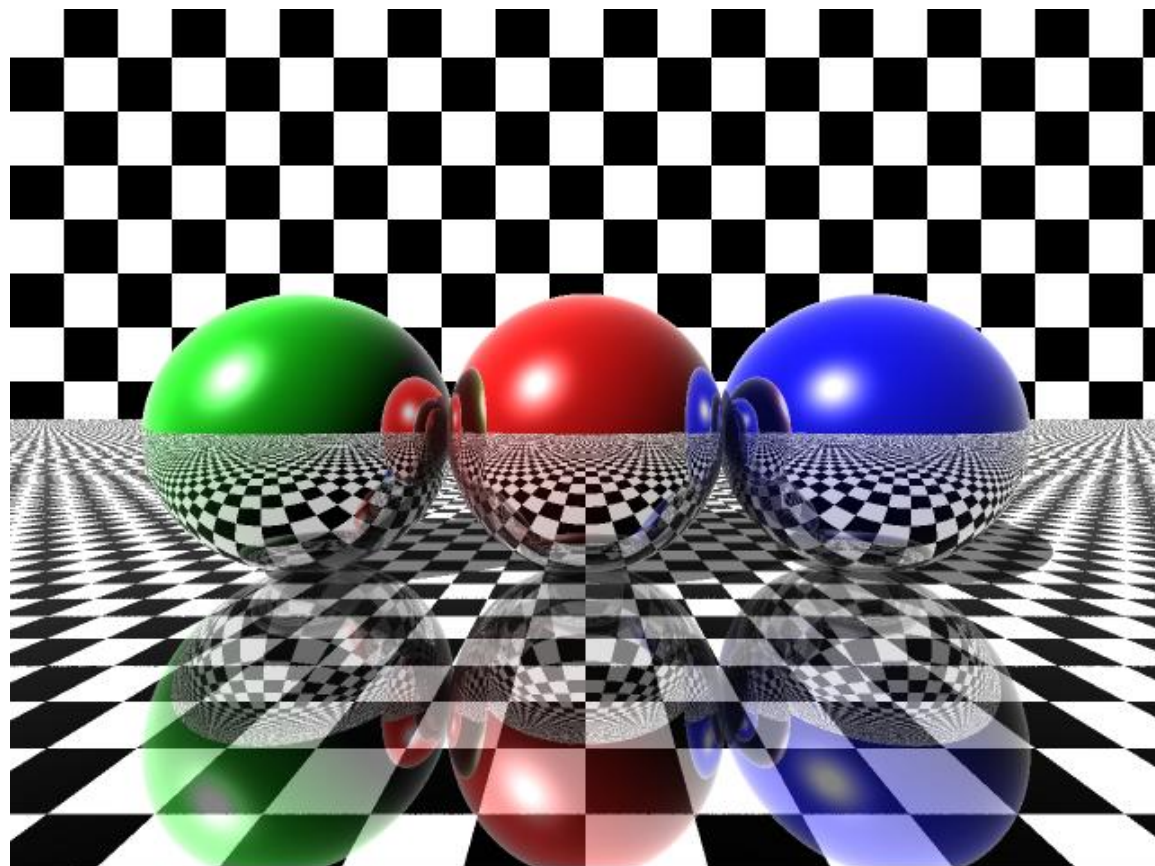


数学 キャリーログ

制作時期	: 二年後期
使用言語	: C++
使用ライブラリ	: DXライブラリ

内積と外積、行列を使った回転を学び、2Dのカプセルの当たり判定を学びました。ゲームとして遊べるように丸太ゲームを製作しました。当たり判定を可視化することで遊びやすくしました。

レイトレーシング



制作時期	: 二年後期
使用言語	: C++
使用ライブラリ	: DXライブラリ

DXライブラリのピクセルごとに色を指定して描画する機能を使用し、古典的レイトレーシングを製作しました。スクリーンの座標からRayを飛ばして、球体、平面との当たり判定をして、最初に当たったオブジェクトの色情報を取得し1ピクセルずつ描画しました。影、鏡面反射、拡散反射などの表現を実装しました

ありがとうございました